COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server ManagementA	**SECTION:**	CPE31S2
DATE PERFORMED:	28/08/2026	**DATE SUBMITTED:**	28/08/2026
NAME:	Damien Jake D. Gestiada	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **Running Elevated Ad hoc Commands**

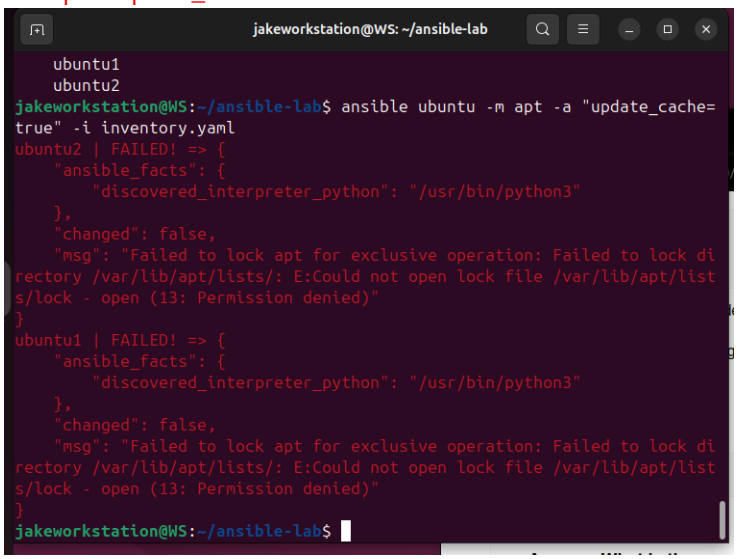
1. PROCEDURE OUTPUT

What procedures did you perform?

Task 1: Run elevated ad hoc commands

1. Locally, we use the command sudo apt update when we want to download package information from all configured resources. The sources are often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following

Command: ansible all -m apt -a update_cache=true



What is the result of the command? Is it successful?

- The command was not successful because updating the package cache requires administrator or root privileges. Ansible connected to the managed hosts, but the normal SSH users did not have sufficient permission to modify the package repository information.

Try editing the command and add something that would elevate the privilege. Issue the command ansible all -m apt -a update_cache=true --become --ask-become-pass. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The update_cache=true is the same thing

as running sudo apt update. The --become command elevates the privileges and the --ask-become-pass asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

```
jakeworkstation@WS:~/ansible-lab$ ansible ubuntu -m apt -a "update_cache=true" -
i inventory.yaml --become --ask-become-pass
BECOME password:
ubuntu1 | FAILED! => {
    "msg": "Missing sudo password"
}
ubuntu2 | FAILED! => {
    "msg": "Missing sudo password"
}
jakeworkstation@WS:~/ansible-lab$ ansible ubuntu -m apt -a "update_cache=true" -
i inventory.yaml --become --ask-become-pass
BECOME password:
```

CHANGED compared to the first command, which is FAILED.

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just change the module part in 1.1 instruction. Here is the command: ansible all -m apt -a name=vim-nox--become --ask-become-pass. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
jakeworkstation@WS:~/ansible-lab$ ansible ubuntu -m apt -a "name=vim-nox state=p
resent" -i inventory.yaml --become --ask-become-pass
BECOME password:
^C [ERROR]: User interrupted execution
jakeworkstation@WS:~/ansible-lab$ cd ..
jakeworkstation@WS:~$ ansible ubuntu -m apt -a "name=vim-nox state=present" -i i
nventory.yaml --become --ask-become-pass
BECOME password:
[WARNING]: Unable to parse /home/jakeworkstation/inventory.yaml as an inventory
source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that
the implicit localhost does not match 'all'
[WARNING]: Could not match supplied host pattern, ignoring: ubuntu
```

a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?

```
jakeworkstation@WS:~$ apt search vm-nox
Sorting... Done
Full Text Search... Done
```

```
jakeworkstation@WS:~$ which vim
/usr/bin/vim
```

```
Last login: Wed Aug 19 17:37:35 2026 from 192.168.56.101
jakeserver@server1:~$ which vim
jakeserver@server1:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security 2:9.1.0016-1ubuntu7.18 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [installe
d,automatic]
  Vi IMproved - enhanced vi editor - compact version
```

```
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Aug 19 17:37:35 2026 from 192.168.56.101
jakeserver2@server2:~$ which vim
jakeserver2@server2:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security 2:9.1.0016-1ubuntu7.18 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.18 amd64 [installe
d,automatic]
  Vi IMproved - enhanced vi editor - compact version
```

- Yes. The command successfully installed or ensured that the vim-nox package was present on both Ubuntu managed nodes. Ansible used elevated privileges to perform the package installation.

b. Check the logs in the servers using the following commands: cd /var/log. After this, issue the command ls, go to the folder apt and open history.log. Describe what you see in the history.log.

```
jakeserver@server1:~$ cd /var/log
jakeserver@server1:/var/log$ ls
alternatives.log        cups              kern.log.2.gz
alternatives.log.1      cups-browsed      kern.log.3.gz
apport.log              dist-upgrade      lastlog
apt                     dmesg             openvpn
auth.log                dmesg.0           private
auth.log.1              dmesg.1.gz        README
auth.log.2.gz           dmesg.2.gz        speech-dispatcher
auth.log.3.gz           dmesg.3.gz        sssd
boot.log                dmesg.4.gz        syslog
boot.log.1              dpkg.log          syslog.1
boot.log.2              dpkg.log.1        syslog.2.gz
boot.log.3              faillog           syslog.3.gz
boot.log.4              fontconfig.log    sysstat
boot.log.5              gdm3              ubuntu-advantage-apt-hook.log
boot.log.6              gpu-manager.log   ufw.log
bootstrap.log           hp                ufw.log.1
btmp                    installer         ufw.log.2.gz
btmp.1                  journal           ufw.log.3.gz
cloud-init.log          kern.log          unattended-upgrades
cloud-init-output.log   kern.log.1        wtmp
jakeserver@server1:/var/log$ cd apt
jakeserver@server1:/var/log/apt$ ls
eipp.log.xz  history.log  history.log.1.gz  term.log  term.log.1.gz
```

```
jakeserver@server1:/var/log/apt$ cat history.log

Start-Date: 2026-08-06  00:17:42
Commandline: /usr/bin/unattended-upgrade
Upgrade: openvpn:amd64 (2.6.19-0ubuntu0.24.04.2, 2.6.19-0ubuntu0.24.04.3)
End-Date: 2026-08-06  00:17:48

Start-Date: 2026-08-06  00:17:52
Commandline: /usr/bin/unattended-upgrade
Upgrade: libpipewire-0.3-common:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), pipewire-pulse:amd64
(1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), pipewire:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), libspa
-0.2-bluetooth:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), gstreamer1.0-pipewire:amd64 (1.0.5-1ub
untu3.2, 1.0.5-1ubuntu3.3), pipewire-audio:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), pipewire-b
in:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), libspa-0.2-modules:amd64 (1.0.5-1ubuntu3.2, 1.0.5-
1ubuntu3.3), libpipewire-0.3-0t64:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), libpipewire-0.3-mod
ules:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ubuntu3.3), pipewire-alsa:amd64 (1.0.5-1ubuntu3.2, 1.0.5-1ub
untu3.3)
End-Date: 2026-08-06  00:17:58
```

- The /var/log/apt/history.log file contains records of package-management operations performed using APT. It records information such as the date and time of the operation.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

```
jakeworkstation@WS:~/ansible-lab$ ansible ubuntu -m apt -a "name=snapd state=present" -i inventor
y.yaml --become --ask-become-pass
BECOME password:
ubuntu2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787132103,
    "cache_updated": false,
    "changed": false
}
ubuntu1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787132106,
    "cache_updated": false,
    "changed": false
}
jakeworkstation@WS:~/ansible-lab$
```

a. Issue the command: ansible all -m apt -a name=snapd --become --ask-become-pass Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

- Yes, the command was successful. Ansible checked the Ubuntu managed servers and ensured that the snapd package was installed.

b. Now, try to issue this command: ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass

```
jakeworkstation@WS:~/ansible-lab$ ansible ubuntu -m apt -a "name=snapd state=latest" -i inventory
.yaml --become --ask-become-pass
BECOME password:
ubuntu2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787132103,
    "cache_updated": false,
    "changed": false
}
ubuntu1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787132106,
    "cache_updated": false,
    "changed": false
}
jakeworkstation@WS:~/ansible-lab$
```

Describe the output of this command. Notice how we added the command state=latest and placed

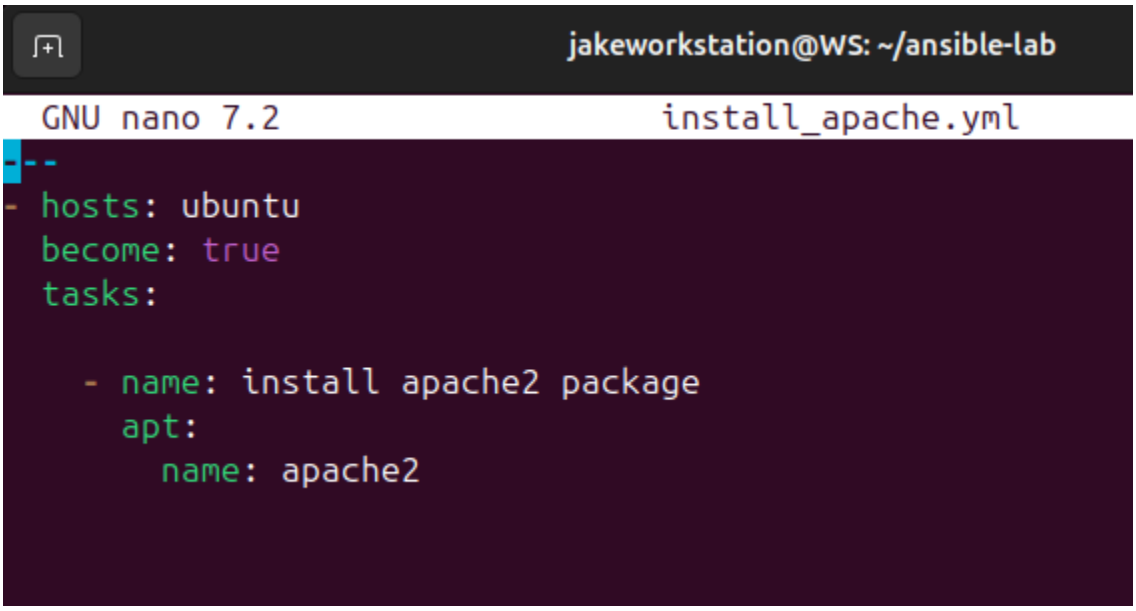
them in double quotations.

- The command checks whether snapd is at the latest available version. If a newer version exists, Ansible updates the package and reports a changed state. If the current version is already the latest, the command succeeds without making changes.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command nano install_apache.yml. This will create a playbook file called install_apache.yaml. The .yaml is the basic standard extension for playbook files.

When the editor appears, type the following:



Make sure to save the file. Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

2. Run the yml file using the command: ansible-playbook --ask-become-pass install_apache.yml. Describe the result of this command.

```
jakeworkstation@WS:~/ansible-lab$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that
the implicit localhost does not match 'all'
[WARNING]: Could not match supplied host pattern, ignoring: ubuntu

PLAY [ubuntu] ******************************************************************
skipping: no hosts matched

PLAY RECAP *********************************************************************

jakeworkstation@WS:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_
apache.yml
BECOME password:

PLAY [ubuntu] ******************************************************************

TASK [Gathering Facts] *********************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [install apache2 package] *************************************************
ok: [ubuntu2]
ok: [ubuntu1]

PLAY RECAP *********************************************************************
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
```

```
jakeworkstation@WS:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_ap
ache.yml
BECOME password:

PLAY [Install Apache and PHP] **************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [ubuntu1]
ok: [ubuntu2]

TASK [Install Apache2] *********************************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [Install PHP support] *****************************************************************
ok: [ubuntu1]
ok: [ubuntu2]

PLAY RECAP *********************************************************************************
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescu
ed=0    ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescu
ed=0    ignored=0

jakeworkstation@WS:~/ansible-lab$
```

jakeworkstation@WS: ~/ansible-lab

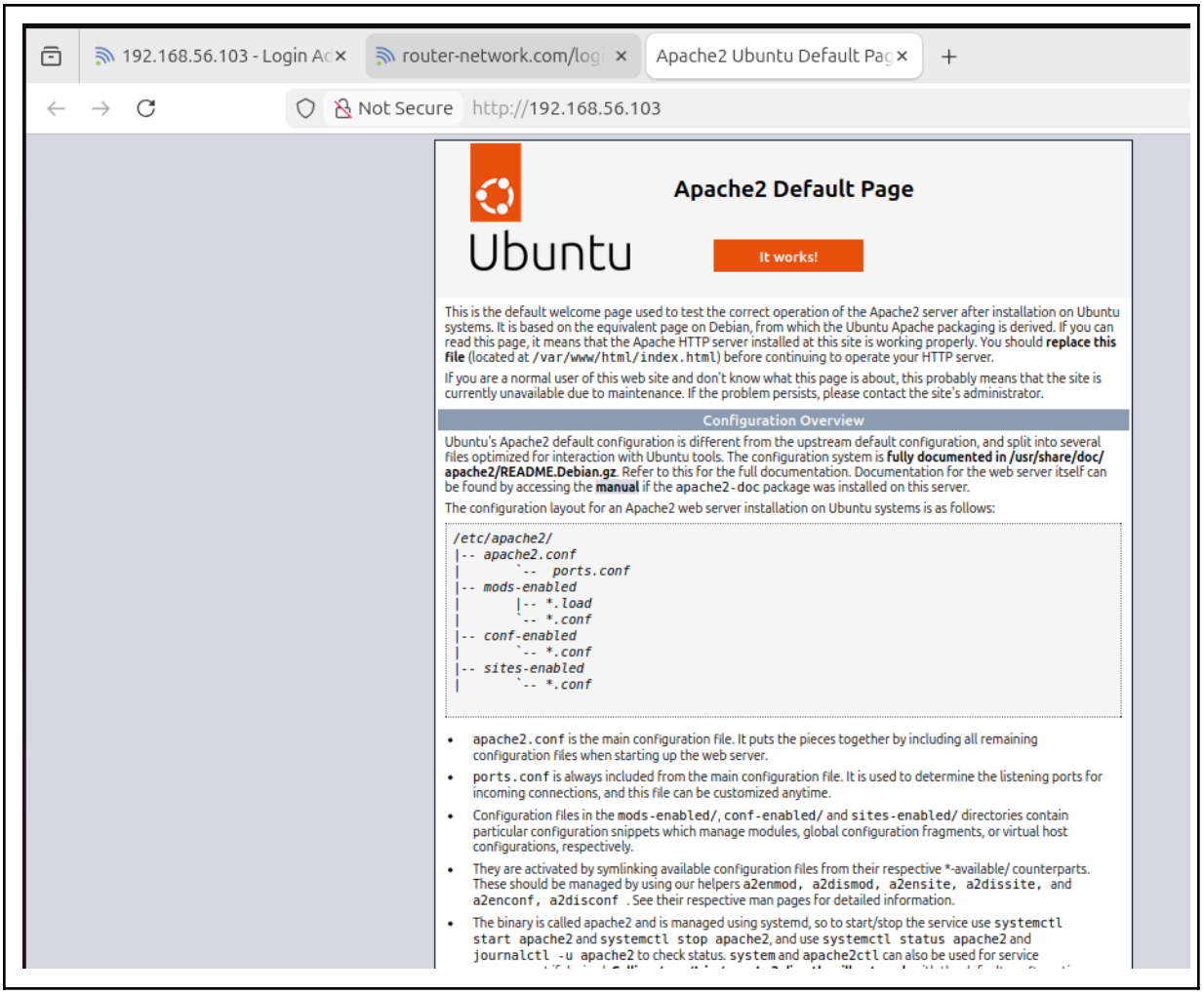
GNU nano 7.2 install_apache.yml

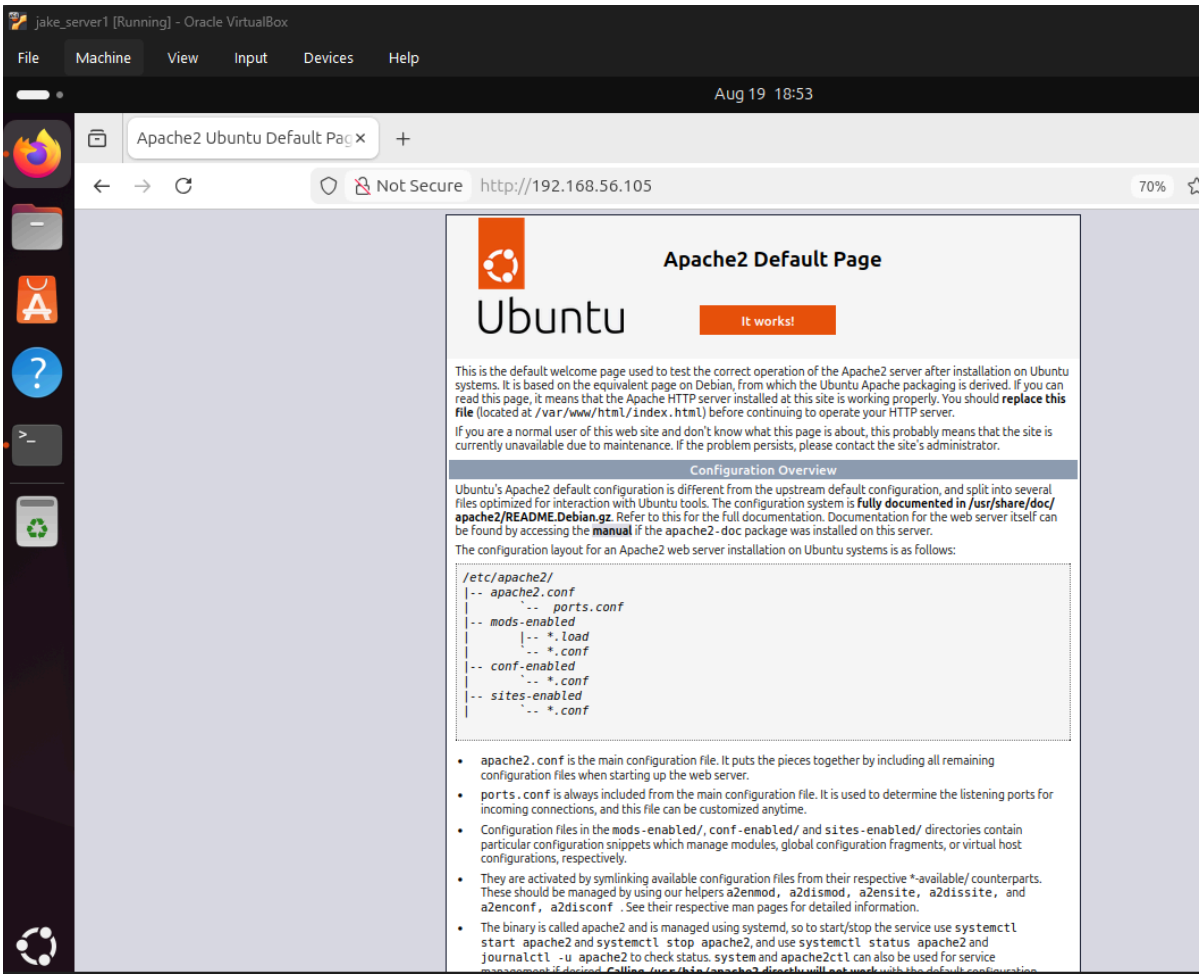
```
---
- name: Install Apache and PHP
  hosts: ubuntu
  become: true

  tasks:
    - name: Install Apache2
      apt:
        name: apache2
        state: present
        update_cache: true

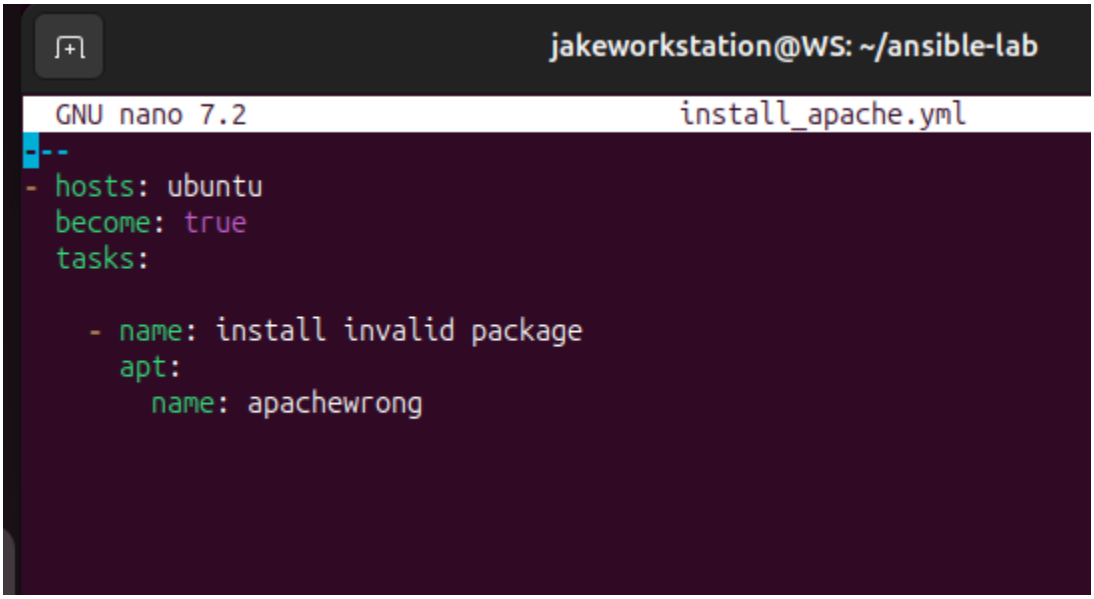
    - name: Install PHP support
      apt:
        name:
          - php
          - libapache2-mod-php
        state: present
```

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.


192.168.56.103 - Login Ac
router-network.com/log
Apache2 Ubuntu Default Pag
Not Secure
http://192.168.56.103
Apache2 Default Page
Ubuntu
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
ports.conf is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
Configuration files in the mods-enabled/, conf-enabled/ and sites-enabled/ directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
They are activated by symlinking available configuration files from their respective *-available/ counterparts. These should be managed by using our helpers a2enmod, a2dismod, a2ensite, a2dissite, and a2enconf, a2disconf . See their respective man pages for detailed information.
The binary is called apache2 and is managed using systemd, so to start/stop the service use systemctl start apache2 and systemctl stop apache2, and use systemctl status apache2 and journalctl -u apache2 to check status. system and apache2ctl can also be used for service



4. Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output?




```
jakeworkstation@WS:~/ansible-lab$ nano install_apache.yml
jakeworkstation@WS:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_
apache.yml
BECOME password:

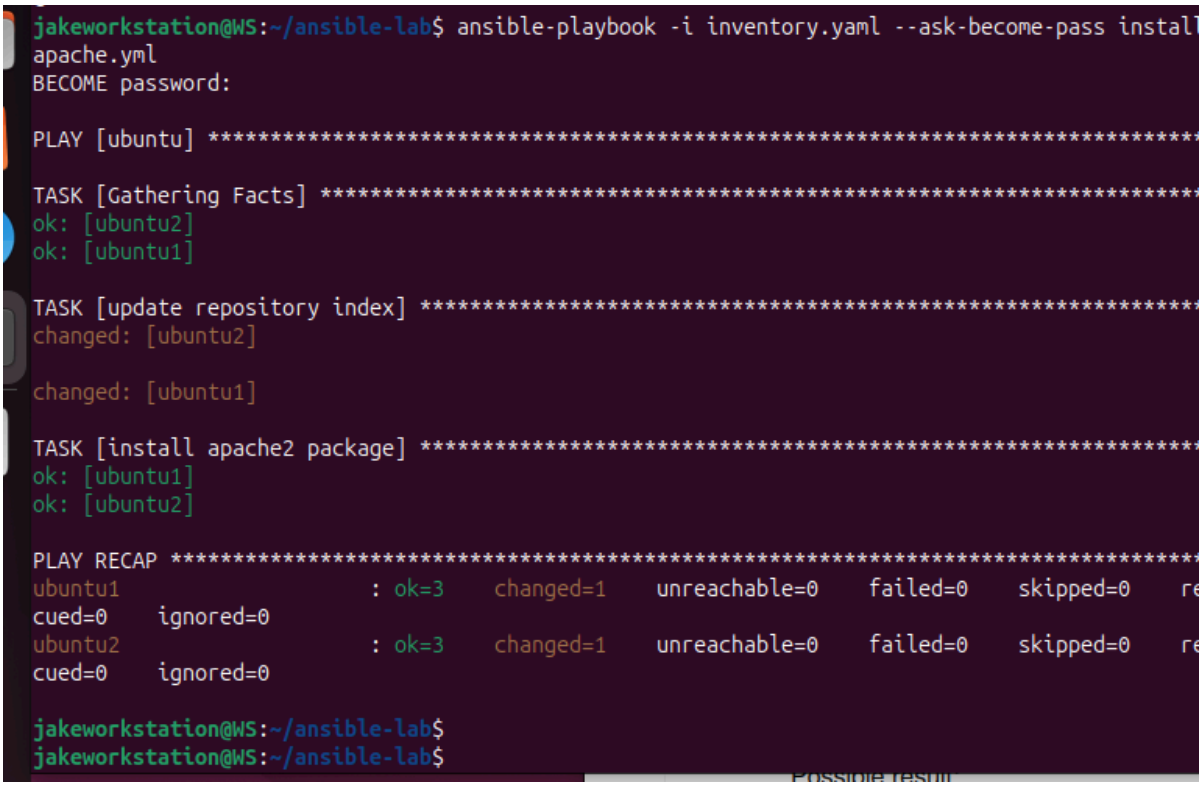
PLAY [ubuntu] ******************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [install invalid package] *************************************************************
fatal: [ubuntu1]: FAILED! => {"changed": false, "msg": "No package matching 'apachewrong' is av
ailable"}
fatal: [ubuntu2]: FAILED! => {"changed": false, "msg": "No package matching 'apachewrong' is av
ailable"}

PLAY RECAP *********************************************************************************
ubuntu1                    : ok=1    changed=0    unreachable=0    failed=1    skipped=0    res
cued=0    ignored=0
ubuntu2                    : ok=1    changed=0    unreachable=0    failed=1    skipped=0    res
cued=0    ignored=0
```

5. This time, we are going to put additional tasks into our playbook. Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.



Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
jakeworkstation@WS:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_
apache.yml
BECOME password:

PLAY [ubuntu] ******************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [update repository index] *************************************************************
changed: [ubuntu2]
changed: [ubuntu1]

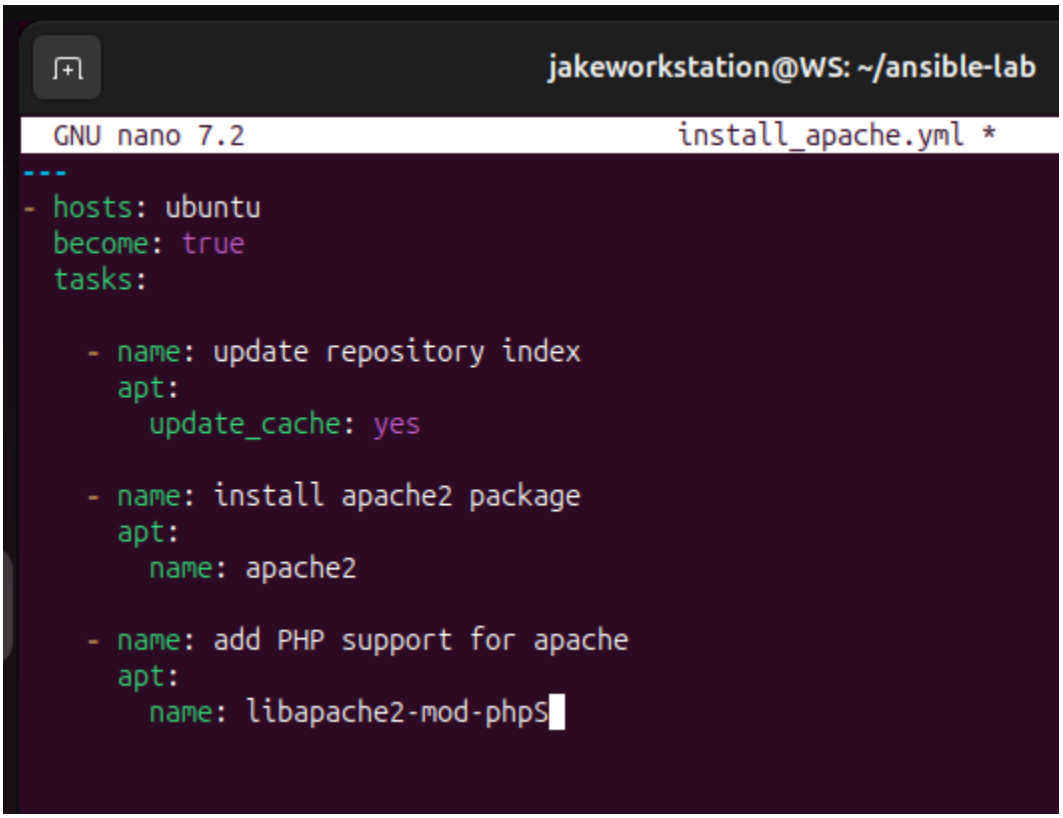
TASK [install apache2 package] *************************************************************
ok: [ubuntu1]
ok: [ubuntu2]

TASK [add PHP support for apache] **********************************************************
ok: [ubuntu2]
ok: [ubuntu1]

PLAY RECAP *********************************************************************************
ubuntu1                    : ok=4    changed=1    unreachable=0    failed=0    skipped=0    res
cued=0    ignored=0
ubuntu2                    : ok=4    changed=1    unreachable=0    failed=0    skipped=0    res
cued=0    ignored=0
```

- The playbook successfully executed the repository-index update and Apache2 installation tasks on both Ubuntu managed servers.

7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.



Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
nothing added to commit but untracked files present (use "git add" to track)
jakeworkstation@WS:~/ansible-lab$ git remote -v
jakeworkstation@WS:~/ansible-lab$ git add install_apache.yml
jakeworkstation@WS:~/ansible-lab$ git commit -m "Add Apache and PHP Ansible playbook"
[master (root-commit) 38f19e3] Add Apache and PHP Ansible playbook
 1 file changed, 16 insertions(+)
 create mode 100644 install_apache.yml
jakeworkstation@WS:~/ansible-lab$ git push
fatal: No configured push destination.
Either specify the URL from the command-line or configure a remote repository using

    git remote add <name> <url>

and then push using the remote name

    git push <name>

jakeworkstation@WS:~/ansible-lab$ git remote add origin https://github.com/DamienJakeGestiada/CPE212_JakeGestiada.git
jakeworkstation@WS:~/ansible-lab$ git remote -v
origin  https://github.com/DamienJakeGestiada/CPE212_JakeGestiada.git (fetch)
origin  https://github.com/DamienJakeGestiada/CPE212_JakeGestiada.git (push)
jakeworkstation@WS:~/ansible-lab$ git branch -M main
jakeworkstation@WS:~/ansible-lab$ git pull origin main --allow-unrelated-histories
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.23 KiB | 630.00 KiB/s, done.
From https://github.com/DamienJakeGestiada/CPE212_JakeGestiada
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
```

```
jakeworkstation@WS:~/ansible-lab$ git pull origin main --allow-unrelated-histories
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.23 KiB | 630.00 KiB/s, done.
From https://github.com/DamienJakeGestiada/CPE212_JakeGestiada
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
jakeworkstation@WS:~/ansible-lab$ git add install_apache.yml inventory.yaml playbook.yaml testvars.yaml challenge.yaml
jakeworkstation@WS:~/ansible-lab$ git commit -m "Add Ansible Activity 4 files"
[main 1ab11ca] Add Ansible Activity 4 files
 4 files changed, 49 insertions(+)
 create mode 100644 challenge.yaml
 create mode 100644 inventory.yaml
 create mode 100644 playbook.yaml
 create mode 100644 testvars.yaml
jakeworkstation@WS:~/ansible-lab$ git push -u origin main
Username for 'https://github.com': Gestiada
Password for 'https://Gestiada@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/DamienJakeGestiada/CPE212_JakeGestiada.git/'
jakeworkstation@WS:~/ansible-lab$
```

- The command can change the APT package index when new or updated package information is downloaded. If the package index was already current, Ansible may report no change.

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository

https://github.com/DamienJakeGestiada/CPE212_JakeGestiada

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Answer the following:

1. **What is the importance of using a playbook?**
   - The reason Ansible playbooks are useful is that they not only organize your tasks but also allow you to define, automate and reuse them all through a YAML file. This is much better than entering the commands one by one manually. A playbook is very useful because it helps to automate the tasks and at the same time can ensure that the set of operations on different servers are done in a consistent and repeatable manner.

2. **Summarize what we have done on this activity.**
   - In this activity, our Ubuntu servers were managed through the use of Ansible by me. I did try executing elevated commands, package installation among others, and also made a playbook that would automatically install Apache and PHP. Besides that, we also gained a little insight into how to verify the outcomes and keeping our code aligned with the repository on GitHub.

Conclusions/Learnings:

- During this exercise, I got a hang of installing and verifying Ansible with a control node and connecting the same to multiple Linux servers by ssh. I was also able to learn about making an inventory file where I was able to separate hosts into groups and metagroups, besides using variables to store such data as OS, and the username of a remote machine I was able to understand that we could use Ansible ad hoc

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For assist error code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.